Casey Barton

Honolulu, HI · (808) 936-5654 · glacier.equilibrium@gmail.com · github.com/GlacierEQ · casey-barton-glaciereq.vercel.app

PROFILE

Forward-deployed AI systems engineer based in Honolulu. I embed with the teams whose workflows I'm deploying — scope the ambiguous process, build the narrow proof, harden what carries load — then feed what the deployment taught me back into the platform.

Three years running an evidence-grade agent control plane: governed GenAI/agent systems with explicit evaluation gates, policy enforcement and approval gates on consequential actions, AI observability (usage, behavior, and failure patterns), MCP-style platform integrations, durable multi-agent workflows with recovery runbooks. What JetBrains Central asks FDEs to deliver in enterprise environments is what I already engineer into every release.

CORE COMPETENCIES

Engineering: Python · TypeScript · Node.js · SQL · Bash · REST APIs · webhooks · Supabase · Postgres · Vercel · GitHub Actions (CI/CD) · containers

AI & Agents: LLM agent frameworks · function calling · RAG · prompt & context engineering · MCP protocol · durable multi-agent workflows · gold sets & eval harnesses · LLM-as-judge

AI Governance & Observability: evaluation with explicit release thresholds · usage metering & audit receipts · error taxonomies · failure analysis & root-cause quarantine · policy enforcement · approval gates on consequential actions · access-gated operations · observability/SLOs · rollback & recovery runbooks

Deployment: demos & POCs · scoped implementations · milestones & implementation plans · delivery frameworks · end-to-end coordination · deployment → platform feedback loops

EXPERIENCE

Founder / Applied AI Systems Architect 2023 – Present

GlacierEQ — Honolulu, HI

- Deploy agent systems from ad-hoc experimentation into governed production: develop, configure, deploy, and optimize GenAI/RAG/agent systems across a **1,398-repository** platform (live-verified 2026-10-19; re-verify before any submit — the estate is actively written to) — agent governance, MCP-style integrations, context externalization, document workflows.
- Evaluation gates every release: the document forge fails closed — any number not on the candidate kit-number allowlist raises **HonestyError** instead of rendering; the campaign engine composes assessments into **GO / NO-GO** with immutable evidence models and stable finding codes. Governance is engineered into the build, not documented around it.
- Run eval harnesses with gold sets and LLM-as-judge (local Mistral API, live-verified 2026-10-06) over retrieval, tool use, and multi-step agent behavior; error taxonomies and per-run failure analysis feed prompt and configuration tuning validated before anything ships.
- AI observability by default: structured logs, task IDs, correlation IDs, retry counts, success/failure metrics — planned / started / succeeded / failed / verified distinguished on every run; every run emits audit receipts with claim tracking and evidence boundaries.

- Policy enforcement on consequential actions: approval gates with typed state transitions and recoverable runs; recovery restricted to contingency evidence declared before reassessment — the engine never silently tunes values until they pass.
- Integrated **65 MCP connectors** plus APIs, webhooks, and data pipelines — typed schemas, claim tracking, evidence boundaries, idempotent automation (lineage 2026-10-08–13; re-verify before any submit). This is the developer-platform and integration surface an FDE owns on the customer side.
- Durable multi-agent workflows: planning with typed handoff contracts, memory across runs, error recovery with root-cause quarantine, human-in-the-loop escalation — the recoverable agent execution pattern that enterprise deployments need underneath.
- Failure recovery with receipts: the stranded-files move worker moved **6,000+ files** Dropbox → Google Drive with md5+size verification per file and per-file root-cause quarantine — quarantined items are analyzed by cause, not retried blindly; nothing deleted from source; idempotent JSONL ledger resume.
- Embed with the operators whose workflows I'm automating: demos and proof-of-concepts, technical requirements, milestones and implementation plans, end-to-end coordination — then harden what the deployment taught me into platform capabilities. The delivery → product feedback loop the FDE role exists for.

EDUCATION

University of Hawaii at Manoa 2016
Bachelor of Science

AWS Cloud Institute 2025
Cloud Computing program

Colorado State University
Undergraduate coursework

SELECTED PROOF

Job-App Helix — attributable campaign engine

- Composes assessments into campaign-level **GO / NO-GO** with immutable evidence models, stable finding codes, and fail-closed decisions.
- Recovery restricted to contingency evidence declared before reassessment — the engine never silently tunes values until they pass.

GlacierEQ estate — governed agent platform

- Durable patterns for claim tracking, evidence boundaries, audit receipts, and human-in-the-loop control across the full estate.
- Eval harnesses with gold sets and LLM-as-judge, adversarial review, and the fail-closed honesty gate — governance is engineered, not documented.

SKILLS

Languages: Python · TypeScript · SQL · Bash · JavaScript / Node.js

AI / Agents: LLM agent frameworks · function calling · RAG · MCP protocol · durable multi-agent workflows · LLM evaluation · prompt & context engineering

Data & Infra: Supabase · Postgres · Vercel · GitHub Actions · REST APIs · webhooks · containers

Practice: demos & POCs · eval gates · error taxonomies · failure analysis · observability/SLOs · audit receipts · idempotent automation · recovery runbooks · delivery frameworks

Evidence boundary: portfolio repositories are independent work. Domain-inspired demonstrations are not claims of company affiliation, production deployment, or certification. No enterprise-scale FDE/consulting engagements, no SOC2/FedRAMP/HIPAA delivery experience, no ex-Principal/ex-Staff/ex-CTO corporate history — the transferable practice is evidence discipline: hash receipts, provenance envelopes, audit trails, fail-closed governance. Java: not on the stack; Python is the working language. Estate counts carried from the most recent verified pack lineage (2026-10-19) — re-verify before any submit; the estate drifts.